

Topic 4: Bit Fields in C

1. High-Yield Conceptual Theory

1.1 Purpose, Syntax & Architectural Need

In C, **bit fields** allow the programmer to allocate fractional byte storage (an exact number of bits) to structure or union members. They are primarily used in:

- **Hardware device registers:** Mapping control/status registers where individual bits represent status flags.
- **Network protocol packet headers:** IP/TCP header fields (e.g., IP version = 4 bits, IHL = 4 bits, flags = 3 bits).
- **Memory-constrained environments:** Packing boolean flags tightly.

General Syntax

```
struct TagName {  
    type [member_name] : width_in_bits;  
    ...  
};
```

- **Permissible Types (C99/C11 standards):** `unsigned int`, `signed int`, `_Bool`, or implementation-defined integer types.

1.2 Mathematical Representation & Two's Complement Range

For a bit field of width w :

Unsigned Range: $[0, 2^w - 1]$

Signed Range (Two's Complement): $[-2^{w-1}, 2^{w-1} - 1]$

The 1-Bit Signed Integer Trap

This is one of the most famous traps in C programming and competitive exams:

```
struct Flag {  
    int f : 1; // Signed 1-bit integer  
};
```

```
struct Flag s;
s.f = 1;
printf("%d", s.f); // OUTPUT: -1 (NOT 1!)
```

• Why does this happen?

- Width $w = 1$.
- If `f` is signed, the single bit is the **sign bit**.
- The range for a 1-bit signed integer is:

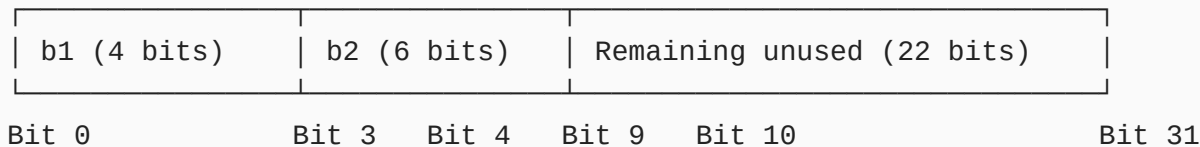
$$[-2^{1-1}, 2^{1-1} - 1] = [-2^0, 2^0 - 1] = [-1, 0]$$

- When binary `1` is stored in a 1-bit signed container, its MSB (and only bit) is `1`, which represents -1 in two's complement.
- To store `0` and `1` safely, it **must** be declared as `unsigned int f : 1;`.

1.3 Packing and Alignment Rules

Bit fields are packed into consecutive bit positions within an **allocation unit** (typically `sizeof(int) = 4` bytes or 32 bits on 32/64-bit systems).

Allocation Unit (32 bits):



1. Adjacent Bit Fields of the Same Type:

- If the adjacent bit field fits within the remaining bits of the current allocation unit, it is packed into the same unit.
- If it **does not fit**, whether it is split across allocation units or moved to the next allocation unit is implementation-defined. Most standard compilers (GCC/Clang on x86/x86-64) **do not split** a bit field across allocation unit boundaries; they start a new unit.

2. Overall Structure Size:

- The total size of the `struct` must be a multiple of the alignment requirement of its largest non-bit-field member or underlying allocation unit type.

1.4 Special Bit-Field Declarations

1. Unnamed Bit Fields (Padding)

An unnamed bit field provides explicit internal padding without exposing a variable name:

```
struct Header {
    unsigned int flag : 1;
    unsigned int      : 3; // Unnamed: skips 3 bits
    unsigned int code : 4;
};
```

- Total bits used = $1 + 3 + 4 = 8$ bits.

2. Zero-Width Bit Fields (: 0)

A zero-width bit field has special semantics in standard C:

```
struct Aligned {
    unsigned int a : 5;
    unsigned int  : 0; // Forces alignment to the NEXT allocation unit boundary
    unsigned int b : 5;
};
```

- **Rules:**
 - It **cannot** have a name.
 - It forces the compiler to terminate the current allocation unit and align the next member to the start of a **fresh allocation unit**.
 - On a 32-bit unsigned int system: a occupies the first 4-byte word; : 0 ends that word; b occupies the second 4-byte word. Total sizeof(struct Aligned) = 8 bytes .

1.5 Strict Language Restrictions on Bit Fields

The C standard imposes strict constraints on bit fields because memory is byte-addressable, not bit-addressable:

1. No Address-of Operator (&):

```
struct S { unsigned int b : 4; } s;
int *p = &s.b; // COMPILE ERROR: cannot take address of bit-field
```

- Pointers point to byte addresses. A single bit has no distinct memory address.

2. No Bit-Field Arrays:

```
struct S { unsigned int arr[10] : 4; }; // COMPILE ERROR
```

3. No `sizeof` on Bit Fields:

```
sizeof(s.b); // COMPILE ERROR: 'sizeof' applied to a bit-field
```

- You can apply `sizeof(s)` to the whole struct, but never to an individual bit-field member.

4. Width Constraints:

- The width w must be an integer constant expression with:

$$0 \leq w \leq \text{Total bits in underlying type}$$

- A width of 0 is only valid for unnamed fields.
- `unsigned int x : 33;` is illegal on a 32-bit `int` system.

2. Common GATE Pitfalls & Edge Cases

1. Sign Extension during Promotion

When reading a signed bit field, it undergoes sign extension to a standard 32-bit signed integer:

```
struct Register {  
    int val : 3; // Range: [-4, 3]  
} r;  
r.val = 5; // Binary: 101 (in 3 bits)
```

- In 3-bit two's complement, binary `101` represents -3 .
- When evaluated in arithmetic or `printf("%d", r.val)`, it sign-extends to `0xFFFFFFFF` (which is -3).

2. Endianness Ambiguity

- While bit widths are well-defined, the order of bits inside a byte (whether bit-fields are allocated Least-Significant-Bit first or Most-Significant-Bit first) is **implementation-defined**.
- On x86 (Little-Endian), bit fields are typically allocated from the low-order bits (LSB) toward high-order bits.

3. Mixing Bit-Field Types with Different Alignment

If a struct contains unsigned short : 4 followed by unsigned int : 4 , compilers may either pack them together or start a new alignment boundary depending on the target architecture's packing rules. GATE questions standardly stick to unsigned int or int to maintain uniformity.

3. Previous Year GATE Questions with Step-by-Step Solutions

PYQ 1 (GATE CS 2008 — Adapted)

Question:

Consider the following C declaration:

```
struct test {
    unsigned int a : 1;
    unsigned int b : 2;
    unsigned int c : 3;
    unsigned int d : 4;
};
```

Assuming an architecture where the allocation unit for unsigned int is 32 bits (4 bytes), what is the value of sizeof(struct test) ?

Step-by-step Solution:

1. Sum the required bit-widths for all members:

$$\text{Total bits} = a + b + c + d = 1 + 2 + 3 + 4 = 10 \text{ bits}$$

2. The underlying type is unsigned int , which has an allocation unit size of 32 bits (4 bytes).
3. Since 10 bits < 32 bits, all four members fit comfortably within a **single** 32-bit allocation unit.
4. Struct size must be a multiple of the alignment of unsigned int (4 bytes).
5. Therefore, the structure requires exactly 1 allocation unit = 4 bytes.

Output: 4 bytes

PYQ 2 (GATE CS 2011 — Adapted Variant)

Question:

What does the following C program print?

```
#include <stdio.h>
struct Bits {
    unsigned int x : 2;
    unsigned int y : 2;
```

```
};

int main() {
    struct Bits b;
    b.x = 5;
    b.y = 2;
    printf("%u %u\n", b.x, b.y);
    return 0;
}
```

Step-by-step Solution:

1. Member `x` has a width of 2 bits (unsigned).
 - Range of `x`: $[0, 2^2 - 1] = [0, 3]$.
 - Binary mask for 2 bits: $2^2 - 1 = 3$ (`0b11`).
2. Assignment `b.x = 5`:
 - Binary representation of 5: `...00000101`.
 - Only the lowest 2 bits are retained: `01` in binary = 1.
 - Alternatively, via unsigned modulo: $5 \pmod{2^2} = 5 \pmod{4} = 1$.
3. Member `y` has a width of 2 bits (unsigned).
 - Assignment `b.y = 2`:
 - Binary of 2: `10`, which fits directly within $[0, 3]$. Thus `b.y = 2`.
4. Output printed: `1 2`.

Output: `1 2`

4. Tricky Practice Questions (Edge Cases & Traps)

Practice Question 1: Unsigned vs Signed 1-bit Field in Branch Conditions

Question:

What will be the output of the following C program?

```
#include <stdio.h>
struct S {
    int a : 1;
    unsigned int b : 1;
};
```

```

int main() {
    struct S s;
    s.a = 1;
    s.b = 1;

    if (s.a == 1)
        printf("A ");
    else
        printf("NOT_A ");

    if (s.b == 1)
        printf("B");
    else
        printf("NOT_B");

    return 0;
}

```

Detailed Explanation:

1. `s.a` is a 1-bit **signed** integer:
 - Setting `s.a = 1` sets its single bit to `1`.
 - As the sign bit is `1`, `s.a` represents `-1`.
 - Comparison: `if (s.a == 1) → if (-1 == 1) → False`.
 - Prints: `NOT_A`.
2. `s.b` is a 1-bit **unsigned** integer:
 - Setting `s.b = 1` stores value `1`.
 - Comparison: `if (s.b == 1) → if (1 == 1) → True`.
 - Prints: `B`.
3. Combined Output: `NOT_A B`.

Output: `NOT_A B`

Practice Question 2: Effect of Zero-Width Bit Fields on `sizeof`

Question:

Consider the following structure on a machine where `sizeof(unsigned int) == 4` bytes:

```

struct Layout {
    unsigned int f1 : 10;
    unsigned int f2 : 12;
    unsigned int   : 0;
    unsigned int f3 : 5;
    unsigned int   : 0;
}

```

```
    unsigned int f4 : 8;
};
```

What is `sizeof(struct Layout)` ?

Detailed Explanation:

1. First allocation unit (Unit 1):

- `f1 : 10 bits`.
- `f2 : 12 bits`.
- Bits used so far = $10 + 12 = 22$ bits 32 bits.

2. Next line: `unsigned int : 0;`

- This is a **zero-width bit field**. It immediately forces the closure of Unit 1 (leaving the remaining 10 bits as padding) and aligns to the start of a fresh allocation unit.

3. Second allocation unit (Unit 2):

- `f3 : 5 bits`.

4. Next line: `unsigned int : 0;`

- Another zero-width bit field. Forces the closure of Unit 2 (leaving 27 bits as padding) and aligns to Unit 3.

5. Third allocation unit (Unit 3):

- `f4 : 8 bits`.
- Remaining 24 bits are padded to complete the 32-bit allocation unit.

6. Total allocation units used: 3 units $\times$ 4 bytes = 12 bytes.

Output: 12

Practice Question 3: Union with Bit Fields (Memory Aliasing)

Question:

What will be the output of the following program on a little-endian system with 32-bit `unsigned int` ?

```
#include <stdio.h>
union Data {
    unsigned int full;
    struct {
        unsigned int b0 : 8;
        unsigned int b1 : 8;
        unsigned int b2 : 8;
        unsigned int b3 : 8;
    } parts;
};
```

```
int main() {
    union Data d;
    d.full = 0x12345678;
    printf("0x%X\n", d.parts.b0);
    return 0;
}
```

Detailed Explanation:

1. The `union` shares memory between `full` (32 bits) and the `struct parts` (which has four 8-bit fields totaling 32 bits).
2. On a **little-endian** system:
 - The least significant byte (LSB) is stored at the lowest memory address.
 - Number `0x12345678` :
 - Address 0 (lowest byte): `0x78`
 - Address 1: `0x56`
 - Address 2: `0x34`
 - Address 3 (highest byte): `0x12`
3. In bit-field layout on standard little-endian systems, `b0` maps to the lowest 8 bits (Address 0).
4. Therefore, `d.parts.b0` reads `0x78`.

Output: `0x78`

5. Quick Revision & Edge-Case Checklist

- ☐ Applying `&` (address-of) to a bit-field member is a **compile-time error**.
- ☐ Applying `sizeof` to an individual bit-field member is a **compile-time error**.
- ☐ Arrays of bit fields are strictly prohibited in C.
- ☐ A 1-bit signed bit field holds values $-1, 0$, never positive $+1$.
- ☐ Unsigned bit fields wrap around modulo 2^w upon overflow ($w = \text{width}$).
- ☐ An unnamed bit field `unsigned int : width;` reserves unused padding bits.
- ☐ An unnamed zero-width bit field `unsigned int : 0;` breaks packing and forces alignment to the next allocation unit boundary.
- ☐ Bit fields cannot exceed the bit-width of their declared integer type.